

Computer Science 323
Fall 2024

Final Project

Group Members:

Omar Younis
Andrew Kang
Alexis Keck

Method Used:

Predictive Parsing Table

Language:

Python

Table of Contents

<i>Original Program (final.txt)</i>	2
<i>Cleaned Program (final24.txt)</i>	2
<i>Translated Program (Python)</i>	3
<i>Original Grammar</i>	4
<i>EBNF Removed</i>	5
<i>Final CFG</i>	7
<i>Members of FIRST and FOLLOW</i>	9
<i>Predictive Parsing Table</i>	10
<i>Preprocessor Code</i>	14
<i>Complier Code</i>	16

Original Program (final.txt)

Below is the original code which makes up the default file “final.txt”.

```
final.txt
1      program f2024;
2      (* This program computes and prints the value
3        of an expression *)
4      var
5      (* declare variables *)
6      a ,      b2a ,      c, bba      : integer ;
7      begin
8          a          = 3 ;
9          b2a =      14 ;
10         c          = 5  ;
11         print ( c ); (* display c *)
12
13         (* compute the value of the expression *)
14         bba = ( b2a + 2 * c ) * a      ;
15         print ( "value=",      bba  ) ; (* print the value of bba*)
16
17     end
```

Cleaned Program (final24.txt)

Below is what the cleaned version of the original program is when extra spaces, extra lines, and comment lines are removed:

```
final24.txt
1      program f2024 ;
2      var
3      a , b2a , c , bba : integer ;
4      begin
5      a = 3 ;
6      b2a = 14 ;
7      c = 5 ;
8      print ( c ) ;
9      bba = ( b2a + 2 * c ) * a ;
10     print ( "value=", bba ) ;
11     end
```

Translated Program (Python)

Below is the translated program that was given into python. The program and it's output is showed below.

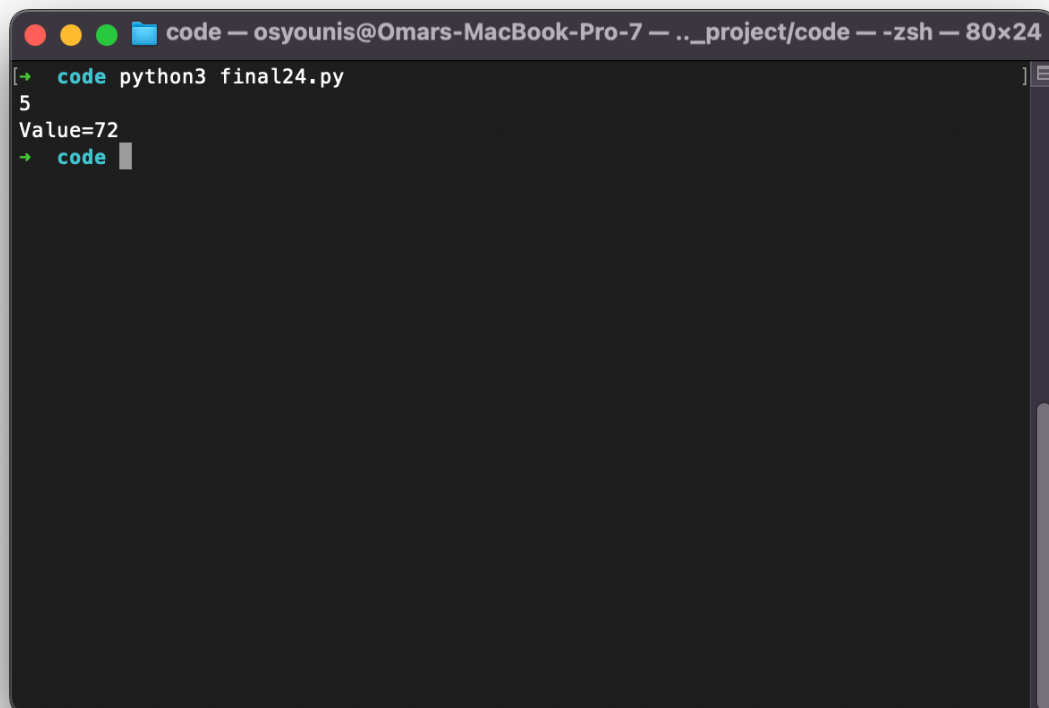
```
"""This module has the original file that the CS 323 Final Project will check,
into Python."""

if __name__ == "__main__":

    a : int = 3
    b2a : int = 14
    c : int = 5

    print(c)
    bba : int = (b2a + (2 * c)) * a

    print(f"Value={bba}")
```

A screenshot of a macOS terminal window. The title bar shows the window name as 'code — osyounis@Omars-MacBook-Pro-7 — .._project/code — -zsh — 80x24'. The terminal content shows a prompt '[→ code' followed by the command 'python3 final24.py'. The output of the script is displayed on two lines: '5' and 'Value=72'. Below the output, there is another prompt '[→ code' with a cursor. The terminal has a dark background with light-colored text.

```
code — osyounis@Omars-MacBook-Pro-7 — .._project/code — -zsh — 80x24
[→ code python3 final24.py
5
Value=72
[→ code
```

Original Grammar

Below is the unaltered grammar for the language in question.

```
<prog>      → program <identifier>; var <dec-list> begin <stat-list> end
<identifier> → <letter>{<letter>|digit}&
<dec-list>  → <dec> : <type> ;
<dec>       → <identifier> , <dec>|<identifier>
<type>      → integer
<stat-list> → <stat> | <stat> <stat-list>
<stat>      → <write> | <assign>
<write>     → print(<str><identifier>);
<str>       → "value=",|λ
<assign>    → <identifier> = <expr> ;
<expr>      → <expr> + <term> | <expr> - <term> | <term>
<term>      → <term> * <factor> | <term> / <factor> | <factor>
<factor>    → <identifier> | <number> | (<expr>)
<number>    → <sign><digit>{<digit>}
<sign>      → +|-|λ
<digit>     → 0|1|2|3|4|5|6|7|8|9
<letter>    → a|b|c|d|l|f
```

EBNF Removed

Below is the grammar in BNF form with all EBNF removed.

<prog>	program <identifier>; var <dec-list> begin <stat-list> end
<identifier>	<letter><X>
<X>	<letter><X>
<X>	<digit><X>
<X>	λ
<dec-list>	<dec> : <type> ;
<dec>	<identifier> , <dec>
<dec>	<identifier>
<type>	integer
<stat-list>	<stat>
<stat-list>	<stat><stat-list>
<stat>	<write>
<stat>	<assign>
<write>	print (<str><identifier>;
<str>	"value=" ,
<str>	λ
<assign>	<identifier> = <expr> ;
<expr>	<expr> + <term>
<expr>	<expr> - <term>
<expr>	<term>
<term>	<term> * <factor>
<term>	<term> / <factor>
<term>	<factor>
<factor>	<identifier>
<factor>	<number>
<factor>	(<expr>)
<number>	<sign><digit><Y>
<Y>	<digit><Y>
<Y>	λ
<sign>	+
<sign>	-
<sign>	λ
<digit>	0
<digit>	1
<digit>	2
<digit>	3
<digit>	4
<digit>	5

<digit>	6
<digit>	7
<digit>	8
<digit>	9
<letter>	a
<letter>	b
<letter>	c
<letter>	d
<letter>	l
<letter>	f

Final CFG

Below is the completed CFG in BNF with all EBNF removed, left recursion removed, and collisions removed. Additionally, identifiers have been replaced with shorthand version.

P	program I ; var B begin G end
I	LX
X	LX
X	DX
X	λ
B	C : K ;
C	IZ
Z	, C
Z	λ
K	integer
G	HV
V	G
V	λ
H	W
H	A
W	print(JI);
J	"value=",
J	λ
A	I = E ;
E	TQ
Q	+TQ
Q	-TQ
Q	λ
T	FR
R	*FR
R	/FR
R	λ
F	I
F	N
F	(E)
N	SDY
Y	DY
Y	λ
S	+
S	-
S	λ
D	0
D	1
D	2

D	3
D	4
D	5
D	6
D	7
D	8
D	9
L	a
L	b
L	c
L	d
L	l
L	f

Members of FIRST and FOLLOW

Below are the members of FIRST and FOLLOW of all non-terminals. In addition, the shorthand versions of the non-terminals are paired with their original names to see which is which.

Original	Symbol	FIRST	FOLLOW
<prog>	P	program	\$
<identifier>	I	a b c d l f	; , :) = * / + -
<X>	X	λ 0 1 2 3 4 5 6 7 8 9 a b c d l f	; , :) = * / + -
<dec-list>	B	a b c d l f	begin
<dec>	C	a b c d l f	:
<Z>	Z	λ ,	:
<type>	K	integer	;
<stat-list>	G	print a b c d l f	end
<V>	V	λ print a b c d l f	end
<stat>	H	print a b c d l f	print a b c d l f end
<write>	W	print	print a b c d l f end
<str>	J	λ "value=",	a b c d l f
<assign>	A	a b c d l f	print a b c d l f end
<expr>	E	(+ - 0 1 2 3 4 5 6 7 8 9 a b c d l f	;))
<expr>'	Q	+ - λ	;))
<term>	T	(+ - 0 1 2 3 4 5 6 7 8 9 a b c d l f	+ - ;)
<term>'	R	* / λ	+ - ;)
<factor>	F	(+ - 0 1 2 3 4 5 6 7 8 9 a b c d l f	* / + - ;)
<number>	N	+ - 0 1 2 3 4 5 6 7 8 9	* / + - ;)
<Y>	Y	λ 0 1 2 3 4 5 6 7 8 9	* / + - ;)
<sign>	S	+ - λ	0 1 2 3 4 5 6 7 8 9
<digit>	D	0 1 2 3 4 5 6 7 8 9	0 1 2 3 4 5 6 7 8 9 a b c d l f * / + - ;) , : =
<letter>	L	a b c d l f	0 1 2 3 4 5 6 7 8 9 a b c d l f * / + - ;) , : =

Predictive Parsing Table

	a	b	c	d	l	f
P						
I	LX	LX	LX	LX	LX	LX
X	LX	LX	LX	LX	LX	LX
B	C:K;	C:K;	C:K;	C:K;	C:K;	C:K;
C	IZ	IZ	IZ	IZ	IZ	IZ
Z						
K						
G	HV	HV	HV	HV	HV	HV
V	G	G	G	G	G	G
H	A	A	A	A	A	A
W						
J	λ	λ	λ	λ	λ	λ
A	I=E;	I=E;	I=E;	I=E;	I=E;	I=E;
E	TQ	TQ	TQ	TQ	TQ	TQ
Q						
T	FR	FR	FR	FR	FR	FR
R						
F	I	I	I	I	I	I
N						
Y						
S						
D						
L	a	b	c	d	l	f

	0	1	2	3	4	5	6	7	8	9
P										
I										
X	DX	DX	DX	DX	DX	DX	DX	DX	DX	DX
B										
C										
Z										
K										
G										
V										
H										
W										
J										
A										
E	TQ	TQ	TQ	TQ	TQ	TQ	TQ	TQ	TQ	TQ
Q										
T	FR	FR	FR	FR	FR	FR	FR	FR	FR	FR
R										
F	N	N	N	N	N	N	N	N	N	N
N	SDY	SDY	SDY	SDY	SDY	SDY	SDY	SDY	SDY	SDY
Y	DY	DY	DY	DY	DY	DY	DY	DY	DY	DY
S	λ	λ	λ	λ	λ	λ	λ	λ	λ	λ
D	0	1	2	3	4	5	6	7	8	9
L										

	+	-	*	/	;	:	(	)	,	=
P										
I										
X	λ	λ	λ	λ	λ	λ		λ	λ	λ
B										
C										
Z						λ			,C	
K										
G										
V										
H										
W										
J										
A										
E	TQ	TQ					TQ			
Q	+TQ	-TQ			λ			λ		
T	FR	FR					FR			
R	λ	λ	*FR	/FR	λ			λ		
F	N	N					(E)			
N	SDY	SDY								
Y	λ	λ	λ	λ	λ			λ		
S	+	-								
D										
L										

	“value=”,	print	integer	program	begin	end	var	\$
P				program I ; var B begin G end				
I								
X								
B								
C								
Z								
K			integer					
G		H V						
V		G				λ		
H		W						
W		print (J I);						
J	“value=”,							
A								
E								
Q								
T								
R								
F								
N								
Y								
S								
D								
L								

Preprocessor Code

Below is the code for the preprocessor name “preprocessor.py”. This code opens the input file and cleans it before it is sent to the compiler to check if it is accepted or rejected.

```
#####
Author: Younis, Omar

Assignment: CPSC 323 Final Project
Due Date: 12/11/2024

Purpose: This program is used to clean a file before is it used by the compiler
         program to see if it is accepted or rejected.

#####

import re

#####
#                               Settings                               #
#####
INPUT_FILE = "final.txt"          # The name of the input file to clean.
OUTPUT_FILE = "final24.txt"       # The name of the cleaned file.

#####
#                               Main Code Loop                         #
#####
if __name__ == "__main__":
    # Open file and get all data in it.
    with open(INPUT_FILE, 'r', encoding='utf-8') as f_obj:
        content = f_obj.read()

    # Using regex, look for all comments in the content of the file and replace
    # the text between it with empty text (removing it).
    content = re.sub(r'\(\(*.*?\)*\)', '', content, flags=re.DOTALL)

    # Removing extra spaces and empty lines in the content after comment removal.
    cleaned_lines = []
    lines = content.splitlines()
    for line in lines:
        if line.strip():
            cleaned_lines.append(re.sub(r'\s+', ' ', line).strip())
```

```
# Adding a new line after every cleaned line before adding the new content
# to the new output file. This is done by joins the list with newline chars.
cleaned_content = '\n'.join(cleaned_lines)

# Writing the cleaned content to the output file.
with open(OUTPUT_FILE, 'w', encoding='utf-8') as f_obj:
    f_obj.write(cleaned_content)
```


Compiler Code

Below is the compiler code. This code is made up of multiple files to run. This opens the cleaned programming file, reads it and checks if it is accepted or rejected by the language.

```
"""
```

```
Author: Younis, Omar
```

```
Assignment: CPSC 323 Final Project
```

```
Due Date: 12/11/2024
```

```
Purpose: This program is used to trace a programming file and checking whether  
it is accepted or reject by the language.
```

```
"""
```

```
from typing import Dict, List, Set, Optional
```

```
#####  
#                                     Classes                                     #  
#####
```

```
class Language:
```

```
    """A class representing a language."""
```

```
    def __init__(self,  
                  starting_state: str,  
                  parsing_table: List[List[Optional[str]]],  
                  indexes: Dict[str, int],  
                  reserved: Set[str]) -> None:
```

```
        self._starting_state = starting_state    # The starting state of the  
language.
```

```
        self._parsing_table = parsing_table      # The language's predictive  
parsing table.
```

```
        self._indexes = indexes                  # The index values for the  
terminals
```

```
                                                # and non-terminals in the  
predictive
```

```
                                                # parsing table.
```

```
        self._reserved_words = reserved         # Hold the list of reserved words.
```

```
    def get_indexes(self) -> Dict[str, int]:
```

```
        """Returns the language's reserved words.
```

```
        Returns:
```

```
        Set[str]: A set of the reserved words.
```

```
    """
```

```

        return self._indexes

def get_control_chars(self, non_terminal: str, terminal: str) -> Optional[str]:
    """Gets the value in the predictive parsing table when provided with a
        non-terminal and a terminal character.

    Args:
        non_terminal (str): The non-terminal character
        terminal (str): The terminal character

    Returns:
        Optional[str]: Returns the value from the predictive parsing table
            given a non-terminal and a terminal.
    """
    non_terminal_index = self._indexes[non_terminal]    # Gets the row value
                                                        # for the non-terminal.

    terminal_index = self._indexes[terminal]            # Gets the column value
                                                        # for the terminal.

    return self._parsing_table[non_terminal_index][terminal_index]

def get_starting_state(self) -> str:
    """Gets the starting state/character of the language.

    Returns:
        str: The starting state/character of the language.
    """
    return self._starting_state

class InputStatement:
    """A class representing a statement to check whether is it accepted or
        rejected by a language."""
    def __init__(self, filename: str) -> None:
        self._accepted = None                                # The status indicating if
                                                            # the statement has been accepted
                                                            # or rejected by a language.

        with open(filename, 'r', encoding='utf-8') as f_obj:
            lines = f_obj.readlines()
            content = [line.strip().split() for line in lines]

        # Puts everything in one list
        content_list = []

```

```

        for line in content:
            for character in line:
                content_list.append(character)

# Storing in class
self._contents = content_list

def set_accepted(self, flag: bool) -> None:
    """Sets the flag indicating whether the statement has been accepted or
    rejected by the language.

    True -> Accepted
    False -> Rejected

    Args:
        flag (bool): What bool you want to set the flag to. True is that the
        statement is accepted and false means it is rejected by the language.
    """
    self._accepted = flag

def get_accepted(self) -> bool:
    """Returns the status of the statement indicating whether or not the
    statement has been accepted or rejected by the language tracing it.

    Returns:
        bool: Returns True if the statement is accepted by the language.
        Returns False if the statement is rejected by the language.
    """
    return self._accepted

def get_statement(self) -> str:
    """Gets the statement that a language is tracing.

    Returns:
        str: The statement that is being traced.
    """
    return self._contents

class Trace:
    """A class representing a Trace given a statement and a language to determine
    if the statement is accepted or rejected by the language.
    """
    def __init__(self, user_input: InputStatement, language: Language) -> None:

```

```

self._user_input = user_input
self._language = language
self._trace_expression()
self._statement_accepted_or_not()

def _trace_expression(self) -> None:
    """Runs a trace on the statement being tested by the language to see if
    it is accepted.
    """
    # Index to track which character we are in the statement.
    i = 0
    statement = self._user_input.get_statement()
    tokens = set(self._language.get_indexes().keys())

    # Setting up our stack before we trace with a $ and the starting state of the
    language.
    stack = ['$ ', self._language.get_starting_state()]

    control_char = ''      # Holds the control character we have from our stack.
    control_val = ''       # Holds the value we get from our predictive parsing
                           # table given our control character and the
                           # character we are in our statement.

    # We loop through every character in our statement via the character index.
    while i < len(statement):
        token = statement[i]          # The character we are on from the
statement.

        if token not in tokens:
            j = 0
            while j < len(token):
                character = token[j]
                control_char = stack.pop()      # Pop the top of our stack for our
control char.

                # If our character equals the character we popped from the stack
then
                # we have a match and can move on to the next character in the
statement
                # we are testing.
                if character == control_char:
                    j += 1
                    continue

                # If we don't have a match, we use our character and our control
char

```

```

        # to find out the value we get from our predictive parsing table
and
        # update our control_val variable to the answer.
        control_val = self._language.get_control_chars(control_char,
character)

        # Now we check our control_val.
        match control_val:

            # If it is None, then we know this statement is rejected by
our
            # language so we can our trace and update our user statement
to
            # indicated that the provided statement was rejected.
            case "":
                self._user_input.set_accepted(False)
                break

            # If our value is "lambda" then we need to pop from our stack
again
            # and continue to check the next control value that new
control
            # character gives us when combined with the character from the
            # statement we are on.
            case "lambda":
                continue

            # If none of the above cases are met, then we take whatever
the
            # value of the control_val is and add each character from that
            # result in reverse order to the stack.
            case _:
                for symbol in reversed(control_val.split()):
                    stack.append(symbol)

        i += 1
        continue
control_char = stack.pop()    # Pop the top of our stack for our control
char.

        # If our character equals the character we popped from the stack then
        # we have a match and can move on to the next character in the statement
        # we are testing.
        if token == control_char:
            i += 1
            continue

        # If we don't have a match, we use our character and our control char
        # to find out the value we get from our predictive parsing table and

```

```

# update our control_val variable to the answer.
control_val = self._language.get_control_chars(control_char, token)

# Now we check our control_val.
match control_val:

    # If it is None, then we know this statement is rejected by our
    # language so we can our trace and update our user statement to
    # indicated that the provided statement was rejected.
    case "":
        self._user_input.set_accepted(False)
        break

    # If our value is "lambda" then we need to pop from our stack again
    # and continue to check the next control value that new control
    # character gives us when combined with the character from the
    # statement we are on.
    case "lambda":
        continue

    # If none of the above cases are met, then we take whatever the
    # value of the control_val is and add each character from that
    # result in reverse order to the stack.
    case _:
        for symbol in reversed(control_val.split()):
            stack.append(symbol)

    # Once we are out of the while loop, we check to see if the stack is empty or
    not. If we get
    # to this stage and the stack is empty, it means our trace was successful and
    the statement
    # is accepted by the language, therefore we update the user_input object's
    accepted flag to
    # True to show it was accepted by the language. If we get to this point and
    there are still
    # items in the stack, it means we needed to break out of the loop early
    because the trace
    # failed and the statement was rejected by the language. So we set the
    user_input object's
    # accepted flag to False to indicate it was rejected by the language.
    if stack.pop() == '$':
        self._user_input.set_accepted(True)
    else:
        self._user_input.set_accepted(False)

def _statement_accepted_or_not(self) -> None:
    """Prints out the statement and whether or not it was accepted by the

```

```

        language the statement is being tested against.
        """
    if self._user_input.get_accepted() is True:
        print("Ready to compile.")
    else:
        print("ERROR: Cannot compile.")

#####
#                                     Main Code Loop                                     #
#####
if __name__ == "__main__":
    # Indexes for the columns and rows for the terminals and non-terminals of
    # the provided language.
    symbol_indexes = {'a': 0, 'P': 0,
                      'b': 1, 'I': 1,
                      'c': 2, 'X': 2,
                      'd': 3, 'B': 3,
                      'l': 4, 'C': 4,
                      'f': 5, 'Z': 5,
                      '0': 6, 'K': 6,
                      '1': 7, 'G': 7,
                      '2': 8, 'V': 8,
                      '3': 9, 'H': 9,
                      '4': 10, 'W': 10,
                      '5': 11, 'J': 11,
                      '6': 12, 'A': 12,
                      '7': 13, 'E': 13,
                      '8': 14, 'Q': 14,
                      '9': 15, 'T': 15,
                      '+': 16, 'R': 16,
                      '-': 17, 'F': 17,
                      '*': 18, 'N': 18,
                      '/': 19, 'Y': 19,
                      ';': 20, 'S': 20,
                      ':': 21, 'D': 21,
                      '(': 22, 'L': 22,
                      ')': 23,
                      ',': 24,
                      '=': 25,
                      '"value="': 26,
                      'print': 27,
                      'integer': 28,
                      'program': 29,
                      'begin': 30,
                      'end': 31,
                      '$': 32,

```



```

[('', '', '', '', '', '', 'S D Y', 'S D Y', 'S D Y', 'S D Y', 'S D Y', 'S D Y',
'S D Y', 'S D Y', 'S D Y', 'S D Y', 'S D Y', 'S D Y', '', '', '', '', '', '', '', ''),
('', '', '', '', '', '', '', 'D Y', 'D Y', 'D Y', 'D Y', 'D Y', 'D Y', 'D Y', 'D
Y', 'D Y', 'D Y', 'lambda', 'lambda', 'lambda', 'lambda', 'lambda', '', '', 'lambda',
'', '', '', '', '', '', '', '', '',)],
[('', '', '', '', '', '', 'lambda', 'lambda', 'lambda', 'lambda', 'lambda',
'lambda', 'lambda', 'lambda', 'lambda', 'lambda', '+', '-', '', '', '', '', ''),
('', '', '', '', '', '', '', '', '', '', '', '', '', '', '',)],
[('', '', '', '', '', '', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', ''),
('', '', '', '', '', '', '', '', '', '', '', '', '', '', '',)],
[('a', 'b', 'c', 'd', 'l', 'f', '', '', '', '', '', '', '', '', '', ''),
('', '', '', '', '', '', '', '', 'print ( J I ) ;', '', '', '', '', '', '')]
]

```

```

reserved_words = {'value=', 'print', 'integer', 'program', 'begin', 'end',
'var'}

```

```

# Creating the language to use for the trace
lang = Language('P', table, symbol_indexes, reserved_words)
file_statement = InputStatement("final24.txt")
Trace(file_statement, lang)

```

A terminal window titled "code — osyounis@Omars-MacBook-Pro-7 — ../project/code — -zsh — 80x24". The window shows the following text:

```

Last login: Tue Dec 10 16:39:39 on ttys003
[→ code python3 compiler.py
Ready to compile.
→ code █

```